

■ Java Stream API

Zero to Hero Guide

Complete Reference with Examples, Use Cases & MNC Interview Patterns

Java 8+

Functional

Lambda

Collectors

Parallel Streams

What's Inside

01 Introduction to Streams

What, Why, and How

02 Stream Creation APIs

of, iterate, generate, range...

03 Intermediate Operations

filter, map, flatMap, sorted, distinct...

04 Terminal Operations

collect, reduce, count, forEach...

05 Collectors Deep Dive

groupingBy, partitioningBy, joining...

06 Optional API

Avoid NullPointerException

07 Parallel Streams

Performance & pitfalls

08 Real-World Use Cases

Employee, Orders, Products

09 MNC Interview Q&A;

Top 20 coding patterns

Chapter 01 — Introduction to Streams

Java Streams (introduced in Java 8) allow you to process sequences of elements in a declarative, functional style. Instead of writing loops and mutable state, you describe **what** you want, not **how** to do it.

Key Characteristics

- **Not a data structure:** Streams don't store data — they carry data from a source through a pipeline.
- **Lazy evaluation:** Intermediate operations are not executed until a terminal operation is called.
- **Consumed once:** A stream can be traversed only once. After terminal op, it's closed.
- **Can be sequential or parallel:** switch with `.parallel()` / `.sequential()`
- **Functional in nature:** Operations produce new streams; original source is unchanged.

Stream Pipeline Anatomy

A Stream pipeline consists of three stages:

Stage	Examples	Description
Source	Collection, Array, Files, I/O	Where data comes from
Intermediate	<code>filter()</code> , <code>map()</code> , <code>sorted()</code>	Transform — returns a Stream (lazy)
Terminal	<code>collect()</code> , <code>count()</code> , <code>forEach()</code>	Trigger — produces a result or side-effect

■ Imperative vs Declarative

```
// Classic imperative approach

List<String> result = new ArrayList<>();

for (String s : list) {

    if (s.startsWith("A")) result.add(s.toUpperCase());

}

// Stream (declarative) approach

List<String> result = list.stream()

    .filter(s -> s.startsWith("A"))

    .map(String::toUpperCase)
```


Chapter 02 — Stream Creation APIs

`Stream.of()` — Create from explicit values

```
Stream<String> s = Stream.of("Java", "Python", "Go");

s.forEach(System.out::println);
```

`Collection.stream()` — From any Collection

```
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);

Stream<Integer> s = list.stream();
```

`Arrays.stream()` — From an array

```
int[] arr = {10, 20, 30};

IntStream s = Arrays.stream(arr);

int sum = s.sum(); // 60
```

`Stream.iterate()` — Infinite ordered stream

```
// First 10 even numbers

Stream.iterate(0, n -> n + 2)

.limit(10)

.forEach(System.out::println);
```

`Stream.generate()` — Infinite unordered stream

```
// 5 random doubles

Stream.generate(Math::random)

.limit(5)

.forEach(System.out::println);
```

`IntStream.range()` — Primitive int range (exclusive end)

```
IntStream.range(1, 6).forEach(System.out::println); // 1-5
```

`IntStream.rangeClosed()` — Primitive int range (inclusive end)

```
IntStream.rangeClosed(1, 5).sum(); // 15
```

`Files.lines()` — Stream lines from a file

```
Stream<String> lines = Files.lines(Path.of("data.txt"));

long count = lines.filter(l -> l.contains("ERROR")).count();
```

`String.chars()` — `IntStream` of character codes

```
"Hello".chars()

.mapToObj(c -> String.valueOf((char)c))

.forEach(System.out::println);
```

`Stream.empty()` — Empty stream

```
Stream<String> empty = Stream.empty();
```

`Stream.concat()` — Merge two streams

```
Stream<String> combined = Stream.concat(

Stream.of("A", "B"), Stream.of("C", "D"));
```

`Stream.builder()` — Build stream incrementally

```
Stream.Builder<String> builder = Stream.builder();

builder.add("One"); builder.add("Two");

Stream<String> s = builder.build();
```


Chapter 03 — Intermediate Operations

Intermediate operations are **lazy** — they don't process data until a terminal operation is invoked. They always return a Stream, enabling method chaining.

filter(Predicate)

Keep elements matching a condition

```
List<Integer> evens = Stream.of(1,2,3,4,5,6)

    .filter(n -> n % 2 == 0)

    .collect(Collectors.toList());

// [2, 4, 6]
```

map(Function)

Transform each element

```
List<String> upper = Stream.of("java", "python")

    .map(String::toUpperCase)

    .collect(Collectors.toList());

// [JAVA, PYTHON]
```

mapToInt / mapToLong / mapToDouble

Map to primitive stream

```
int[] lengths = Stream.of("Java", "Go", "Kotlin")

    .mapToInt(String::length)

    .toArray();

// [4, 2, 6]
```

flatMap(Function)

Flatten nested collections

```
List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));

List<Integer> flat = nested.stream()

.flatMap(Collection::stream)

.collect(Collectors.toList());

// [1, 2, 3, 4]
```

distinct()

Remove duplicates (uses equals)

```
Stream.of(1,2,2,3,3,3).distinct()

.forEach(System.out::print);

// 1 2 3
```

sorted()

Natural order sort

```
Stream.of(5,3,1,4,2).sorted()

.forEach(System.out::print);

// 1 2 3 4 5
```

sorted(Comparator)

Custom sort

```
employees.stream()

.sorted(Comparator.comparing(Employee::getSalary).reversed())

.collect(Collectors.toList());
```

peek(Consumer)

Debug / side-effect without consuming

```
Stream.of(1,2,3)
```



```
.peek(n -> System.out.println("Before: " + n))

.map(n -> n * 2)

.peek(n -> System.out.println("After: " + n))

.collect(Collectors.toList());
```

limit(n)

Truncate stream to n elements

```
Stream.iterate(1, n -> n+1).limit(5)

.forEach(System.out::print); // 1 2 3 4 5
```

skip(n)

Skip first n elements

```
Stream.of(1,2,3,4,5).skip(2)

.forEach(System.out::print); // 3 4 5
```

takeWhile(Predicate)

Java 9+ Take while condition is true

```
Stream.of(1,2,3,4,5,1,2)

.takeWhile(n -> n < 4)

.forEach(System.out::print); // 1 2 3
```

dropWhile(Predicate)

Java 9+ Drop while condition is true

```
Stream.of(1,2,3,4,5)

.dropWhile(n -> n < 3)

.forEach(System.out::print); // 3 4 5
```

mapMulti() (Java 16+)

Replace element with zero or more

```
Stream.of(1, 2, 3)

.<Integer>mapMulti((n, consumer) -> {

    consumer.accept(n);

    consumer.accept(n * 10);

})

.forEach(System.out::print); // 1 10 2 20 3 30
```

Chapter 04 — Terminal Operations

Terminal operations **trigger the pipeline** and produce a result or side-effect. After a terminal operation, the stream is **consumed** and cannot be reused.

collect(Collector)

Accumulate into a collection

```
List<String> list = stream.collect(Collectors.toList());

Set<String> set = stream.collect(Collectors.toSet());

Map<K,V> map = stream.collect(Collectors.toMap(k, v));
```

forEach(Consumer)

Side-effect for each element

```
stream.forEach(System.out::println);

// Use forEachOrdered() to preserve order in parallel streams
```

count()

Number of elements

```
long count = stream.filter(s -> s.length() > 3).count();
```

findFirst()

First element (Optional)

```
Optional<String> first = stream.filter(s -> s.startsWith("A")).findFirst();

first.ifPresent(System.out::println);
```

findAny()

Any element — faster in parallel

```
Optional<String> any = stream.filter(s -> s.length() > 5).findAny();
```

anyMatch(Predicate)

True if any element matches

```
boolean hasAdult = persons.stream().anyMatch(p -> p.getAge() >= 18);
```

allMatch(Predicate)

True if ALL elements match

```
boolean allAdults = persons.stream().allMatch(p -> p.getAge() >= 18);
```

noneMatch(Predicate)

True if NO element matches

```
boolean noMinors = persons.stream().noneMatch(p -> p.getAge() < 18);
```

min(Comparator)

Minimum element

```
Optional<Integer> min = Stream.of(3,1,4,1,5).min(Integer::compareTo);
```

max(Comparator)

Maximum element

```
Optional<Employee> topEarner = employees.stream()  
  
    .max(Comparator.comparing(Employee::getSalary));
```

reduce(identity, accumulator)

Fold into a single value

```
int sum = Stream.of(1,2,3,4,5).reduce(0, Integer::sum); // 15  
  
// Without identity – returns Optional  
  
Optional<Integer> product = Stream.of(1,2,3,4).reduce((a,b) -> a * b);
```

toArray()

Convert to Object[]

```
Object[] arr1 = stream.toArray();
```

```
String[] arr2 = stream.toArray(String[]::new);
```

sum / average / summaryStatistics (primitives)

Numeric aggregations

```
IntStream is = IntStream.of(10, 20, 30, 40, 50);

int sum = is.sum(); // 150

double avg = IntStream.of(1,2,3).average().getAsDouble(); // 2.0

IntSummaryStatistics stats = IntStream.of(1,2,3,4,5)

.summaryStatistics();

// count=5, sum=15, min=1, max=5, average=3.0
```

Chapter 05 — Collectors Deep Dive

Collectors is one of the most powerful utility classes in the Stream API. It provides various reduction strategies to accumulate stream elements.

Collectors.toList() / toSet() / toUnmodifiableList()

Basic collection

```
List<String> list = stream.collect(Collectors.toList());

Set<String> set = stream.collect(Collectors.toSet());

List<String> unmod = stream.collect(Collectors.toUnmodifiableList());
```

Collectors.toMap(keyMapper, valueMapper)

Build a Map

```
Map<Integer,String> idToName = employees.stream()

.collect(Collectors.toMap(

Employee::getId,

Employee::getName

));

// Handle duplicate keys with merge function

Map<String,Integer> nameToAge = people.stream()

.collect(Collectors.toMap(

Person::getName,

Person::getAge,

(existing, replacement) -> existing

));
```

Collectors.groupingBy(classifier)

Group into Map>

```
Map<String, List<Employee>> byDept = employees.stream()

.collect(Collectors.groupingBy(Employee::getDepartment));

// Nested grouping: dept -> gender -> employees

Map<String, Map<String, List<Employee>>> nested = employees.stream()

.collect(Collectors.groupingBy(Employee::getDepartment,

Collectors.groupingBy(Employee::getGender))));
```

Collectors.groupingBy with downstream Collector

Count / Sum / Avg per group

```
// Count per dept

Map<String, Long> countByDept = employees.stream()

.collect(Collectors.groupingBy(Employee::getDepartment,

Collectors.counting()));

// Average salary per dept

Map<String, Double> avgSal = employees.stream()

.collect(Collectors.groupingBy(Employee::getDepartment,

Collectors.averagingDouble(Employee::getSalary)));

// Sum of salary per dept

Map<String, Integer> sumSal = employees.stream()

.collect(Collectors.groupingBy(Employee::getDepartment,

Collectors.summingInt(Employee::getSalary)));
```

Collectors.partitioningBy(Predicate)

Split into true/false Map

```
Map<Boolean, List<Employee>> partition = employees.stream()

    .collect(Collectors.partitioningBy(

        e -> e.getSalary() > 50000

    ));

List<Employee> highEarners = partition.get(true);

List<Employee> lowEarners = partition.get(false);
```

Collectors.joining()

Concatenate strings

```
String csv = Stream.of("Java", "Python", "Go")

    .collect(Collectors.joining(", "));

// "Java, Python, Go"

String wrapped = Stream.of("A", "B", "C")

    .collect(Collectors.joining(", ", "[", "]"));

// "[A, B, C]"
```

Collectors.counting()

Count elements

```
long count = stream.collect(Collectors.counting());
```

Collectors.summarizingInt/Long/Double()

Statistics object

```
IntSummaryStatistics stats = employees.stream()

    .collect(Collectors.summarizingInt(Employee::getSalary));

System.out.println(stats.getMax());
```



```
System.out.println(stats.getAverage());
```

Collectors.mapping(mapper, downstream)

Transform before collecting

```
// Get names grouped by department

Map<String, List<String>> deptNames = employees.stream()

.collect(Collectors.groupingBy(

Employee::getDepartment,

Collectors.mapping(Employee::getName, Collectors.toList())

));
```

Collectors.collectingAndThen(downstream, finisher)

Post-process the result

```
// Collect to unmodifiable list

List<String> unmod = stream.collect(

Collectors.collectingAndThen(

Collectors.toList(),

Collections::unmodifiableList

));

// Get count as int (not Long)

int count = stream.collect(

Collectors.collectingAndThen(

Collectors.counting(), Long::intValue

));
```

Collectors.teeing() — Java 12+

Combine two collectors into one result

```
record MinMax(int min, int max) {}

MinMax result = Stream.of(3,1,4,1,5,9,2,6)

.collect(Collectors.teeing(

Collectors.minBy(Integer::compareTo),

Collectors.maxBy(Integer::compareTo),

(min, max) -> new MinMax(min.get(), max.get())

));

// MinMax[min=1, max=9]
```

■ `groupingBy + counting()` is the most common Stream question in MNC interviews. Practice: employee count per dept, top earner per dept, dept with most employees.

Chapter 06 — Optional API

`Optional<T>` is a container that may or may not contain a value. It eliminates `NullPointerException` by making null-handling explicit and expressive.

Creating Optional

```
Optional<String> present = Optional.of("Java"); // throws NPE if null

Optional<String> nullable = Optional.ofNullable(null); // safe

Optional<String> empty = Optional.empty();
```

Checking & Getting

```
optional.isPresent() // true if value exists

optional.isEmpty() // Java 11 – true if empty

optional.get() // get value (throw if empty – avoid!)
```

Safe value access

```
optional.orElse("default") // value or default

optional.orElseGet(() -> compute()) // lazy default

optional.orElseThrow() // throw NoSuchElementException

optional.orElseThrow(RuntimeException::new)
```

Transformations

```
// map – transform if present

Optional<Integer> length = Optional.of("Hello").map(String::length);

// flatMap – when mapper returns Optional

Optional<String> city = Optional.ofNullable(person)
```

```

.flatMap(p -> Optional.ofNullable(p.getAddress()))

.map(Address::getCity);

// filter

Optional<String> longName = Optional.of("Mahindra")

.filter(s -> s.length() > 5);

```

Side effects

```

optional.ifPresent(System.out::println);

// Java 9 – ifPresent or else

optional.ifPresentOrElse(

    val -> System.out.println("Found: " + val),

    () -> System.out.println("Not found")

);

```

Optional with Streams (Java 9+)

```

// stream() converts Optional to Stream of 0 or 1 elements

List<Optional<String>> opts = List.of(

    Optional.of("Java"), Optional.empty(), Optional.of("Go")

);

List<String> values = opts.stream()

    .flatMap(Optional::stream)

    .collect(Collectors.toList());

// [Java, Go]

```

Real-World Chain

```
String city = Optional.ofNullable(order)

    .map(Order::getCustomer)

    .map(Customer::getAddress)

    .map(Address::getCity)

    .orElse("Unknown City");
```

■■ Never use Optional as a method parameter or field. Use it only as a return type to indicate a value might be absent. Avoid optional.get() without isPresent() check.

Chapter 07 — Parallel Streams

Parallel streams use the **Fork/Join framework** to split the workload across multiple CPU cores. They can dramatically speed up CPU-intensive operations on large datasets — but come with caveats.

Creating Parallel Streams

```
// Method 1: from collection

list.parallelStream()

// Method 2: convert sequential to parallel

list.stream().parallel()

// Method 3: convert parallel back to sequential

list.parallelStream().sequential()
```

Performance Example

```
long start = System.currentTimeMillis();

// Sequential

long sumSeq = LongStream.rangeClosed(1, 100_000_000L).sum();

// ~300ms

// Parallel

long sumPar = LongStream.rangeClosed(1, 100_000_000L).parallel().sum();

// ~80ms (4-core machine)
```

When to Use Parallel Streams

■ Use When

Large dataset (100k+ elements)

■ Avoid When

Small collections

CPU-bound computations	I/O bound operations
Stateless, independent operations	Operations with shared mutable state
No ordering requirement	Order matters and must be preserved
No synchronization needed	Using non-thread-safe data structures

Common Pitfall — Thread-Unsafe Code

```
// ■ WRONG – shared mutable state!

List<Integer> result = new ArrayList<>();

Stream.of(1,2,3,4,5).parallel()

    .forEach(result::add); // ConcurrentModificationException risk


// ■ CORRECT – use thread-safe collect

List<Integer> result = Stream.of(1,2,3,4,5).parallel()

    .collect(Collectors.toList());
```

■ Parallel streams use `ForkJoinPool.commonPool()` which has (CPU cores - 1) threads. For I/O tasks, create a custom `ForkJoinPool` instead.

Chapter 08 — Real-World Use Cases

These are the exact types of problems asked in MNC coding rounds. Each example uses a realistic domain model.

Domain Model — Employee

```
record Employee(int id, String name, String dept,  
  
double salary, int age, String gender) {}  
  
List<Employee> employees = List.of(  
  
new Employee(1, "Arun", "IT", 80000, 28, "M"),  
  
new Employee(2, "Priya", "HR", 55000, 32, "F"),  
  
new Employee(3, "Kiran", "IT", 95000, 35, "M"),  
  
new Employee(4, "Sneha", "HR", 60000, 27, "F"),  
  
new Employee(5, "Rahul", "IT", 75000, 30, "M"),  
  
new Employee(6, "Divya", "Finance", 70000, 29, "F"),  
  
new Employee(7, "Suresh", "Finance", 85000, 40, "M"),  
  
new Employee(8, "Lakshmi", "IT", 92000, 33, "F")  
  
);
```

UC-01 — Second Highest Salary

```
Optional<Double> secondHighest = employees.stream()  
  
    .map(Employee::getSalary)  
  
    .distinct()  
  
    .sorted(Comparator.reverseOrder())  
  
    .skip(1)
```



```
.findFirst();

// 92000.0
```

UC-02 — Top 3 Earners per Department

```
Map<String, List<Employee>> top3PerDept = employees.stream()

.collect(Collectors.groupingBy(

Employee::getDept,

Collectors.collectingAndThen(

Collectors.toList(),

list -> list.stream()

.sorted(Comparator.comparingDouble(Employee::getSalary).reversed())

.limit(3)

.collect(Collectors.toList())

)

));
```

UC-03 — Department with highest average salary

```
employees.stream()

.collect(Collectors.groupingBy(

Employee::getDept,

Collectors.averagingDouble(Employee::getSalary)

))

.entrySet().stream()

.max(Map.Entry.comparingByValue())

.map(Map.Entry::getKey)
```

```
.ifPresent(System.out::println);

// IT
```

UC-04 — Count employees per department

```
Map<String, Long> countPerDept = employees.stream()

.collect(Collectors.groupingBy(

Employee::getDept, Collectors.counting()

));

// {IT=4, HR=2, Finance=2}
```

UC-05 — Names of employees earning above average salary

```
double avg = employees.stream()

.mapToDouble(Employee::getSalary).average().orElse(0);

List<String> aboveAvg = employees.stream()

.filter(e -> e.getSalary() > avg)

.map(Employee::getName)

.sorted()

.collect(Collectors.toList());
```

UC-06 — Partition employees by salary > 70k

```
Map<Boolean, List<String>> partitioned = employees.stream()

.collect(Collectors.partitioningBy(

e -> e.getSalary() > 70000,

Collectors.mapping(Employee::getName, Collectors.toList())
```

```
));

// true -> [Arun, Kiran, Rahul, Suresh, Lakshmi, Divya]

// false -> [Priya, Sneha]
```

UC-07 — Comma-separated list of IT employees

```
String itEmployees = employees.stream()

    .filter(e -> e.getDept().equals("IT"))

    .map(Employee::getName)

    .collect(Collectors.joining(", "));

// Arun, Kiran, Rahul, Lakshmi
```

UC-08 — Salary statistics per department

```
Map<String, DoubleSummaryStatistics> stats = employees.stream()

    .collect(Collectors.groupingBy(

        Employee::getDept,

        Collectors.summarizingDouble(Employee::getSalary)

    ));

stats.forEach((dept, s) ->

    System.out.printf("%s: min=%.0f max=%.0f avg=%.0f%n",

        dept, s.getMin(), s.getMax(), s.getAverage()));
```

Chapter 09 — String & Collection Patterns

Frequency of characters in a string

```
String s = "programming";

Map<Character, Long> freq = s.chars()

    .mapToObj(c -> (char) c)

    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));

// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

First non-repeating character

```
Optional<Character> first = s.chars()

    .mapToObj(c -> (char) c)

    .collect(Collectors.groupingBy(Function.identity(),

        LinkedHashMap::new, Collectors.counting()))

    .entrySet().stream()

    .filter(e -> e.getValue() == 1)

    .map(Map.Entry::getKey)

    .findFirst();

// p
```

Find duplicate words in a sentence

```
String sentence = "java is great and java is fun";

Set<String> duplicates = Arrays.stream(sentence.split(" "))

    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))

    .entrySet().stream()
```

```
.filter(e -> e.getValue() > 1)

.map(Map.Entry::getKey)

.collect(Collectors.toSet());

// [java, is]
```

Reverse each word in a sentence

```
String result = Arrays.stream("Hello World Java".split(" "))

.map(w -> new StringBuilder(w).reverse().toString())

.collect(Collectors.joining(" "));

// olleH dlrow avaJ
```

Check if list contains duplicates

```
List<Integer> nums = List.of(1, 2, 3, 2, 5);

boolean hasDupes = nums.stream()

.collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))

.values().stream()

.anyMatch(count -> count > 1);

// true
```

Find numbers divisible by 3 and 5

```
List<Integer> result = IntStream.rangeClosed(1, 100)

.filter(n -> n % 3 == 0 && n % 5 == 0)

.boxed()

.collect(Collectors.toList());

// [15, 30, 45, 60, 75, 90]
```

Sort a Map by value

```
Map<String, Integer> scores = Map.of("Alice",85, "Bob",92, "Carol",78);

scores.entrySet().stream()

.sorted(Map.Entry.<String,Integer>comparingByValue().reversed())

.forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));

// Bob:92, Alice:85, Carol:78
```

Fibonacci using Stream.iterate

```
Stream.iterate(new int[]{0,1}, f -> new int[]{f[1], f[0]+f[1]})

.limit(10)

.map(f -> f[0])

.forEach(n -> System.out.print(n + " "));

// 0 1 1 2 3 5 8 13 21 34
```

Flatten and collect words from list of sentences

```
List<String> sentences = List.of("Java is fast", "Streams are powerful");

List<String> words = sentences.stream()

.flatMap(s2 -> Arrays.stream(s2.split(" ")))

.map(String::toLowerCase)

.distinct()

.sorted()

.collect(Collectors.toList());
```

Chapter 10 — MNC Interview Q&A;

Q1: What is the difference between map() and flatMap()?

map() transforms each element to exactly one output element. flatMap() transforms each element to zero or more elements (another stream) and flattens the result.

```
Stream.of("hello", "world")

.map(s -> s.split("")) // Stream<String[]>

.flatMap(Arrays::stream) // Stream<String>

.distinct()

.forEach(System.out::print); // h e l o w r d
```

Q2: Difference between findFirst() and findAny()?

findFirst() always returns the first element. findAny() may return any element — it's faster in parallel streams as it doesn't force ordering.

```
// Sequential – both return "Java"

Stream.of("Java", "Go", "Kotlin").findFirst();

Stream.of("Java", "Go", "Kotlin").findAny();

// Parallel – findAny() may return any element

Stream.of("Java", "Go", "Kotlin").parallel().findAny();
```

Q3: Difference between Intermediate and Terminal operations?

Intermediate ops are lazy — they return a Stream and are only executed when a terminal op is called. Terminal ops trigger the pipeline and produce a result or side-effect.

```
// Nothing executes until collect() is called!

Stream<Integer> s = Stream.of(1,2,3)

.filter(n -> { System.out.println("filter:" + n); return n > 1; })
```

```
.map(n -> { System.out.println("map:" + n); return n * 2; });

// Terminal triggers execution:

List<Integer> result = s.collect(Collectors.toList());
```

Q4: Can a Stream be reused after a terminal operation?

No. Once a terminal operation is called, the stream is consumed and closed. Attempting to reuse it throws `IllegalStateException`.

```
Stream<String> s = Stream.of("a", "b", "c");

s.forEach(System.out::println); // OK

s.count(); // throws IllegalStateException: stream has already been operated upon
```

Q5: How to get distinct elements sorted in reverse order?

```
Stream.of(5,3,1,3,4,5,2)

.distinct()

.sorted(Comparator.reverseOrder())

.forEach(System.out::print);

// 5 4 3 2 1
```

Q6: `reduce()` vs `collect()` — when to use which?

`reduce()` is for immutable aggregation (produces a single scalar value). `collect()` is for mutable reduction (builds collections, maps, strings).

```
// reduce – scalar

int sum = Stream.of(1,2,3,4,5).reduce(0, Integer::sum);

// collect – mutable container

List<Integer> list = Stream.of(1,2,3)

.collect(Collectors.toList());
```


Q7: How does lazy evaluation benefit Stream performance?

With lazy evaluation, JVM can fuse multiple operations into a single pass and short-circuit as soon as the result is found — avoiding unnecessary processing.

```
// Only processes elements until one is found – no full traversal!

Optional<String> first = Stream.iterate(1, n -> n+1)

    .map(n -> "item" + n)

    .filter(s -> s.endsWith("5"))

    .findFirst(); // stops at "item5"
```

Q8: Write a custom Collector using Collector.of()

```
// Custom Collector to join with | separator

Collector<String, StringBuilder, String> pipeJoiner =

    Collector.of(

        StringBuilder::new, // supplier

        (sb, s) -> { // accumulator

            if (sb.length() > 0) sb.append("|");

            sb.append(s);

        },

        StringBuilder::append, // combiner (parallel)

        StringBuilder::toString // finisher

    );

String result = Stream.of("A","B","C").collect(pipeJoiner);

// A|B|C
```

Quick Reference Cheat Sheet

Operation	Method	Returns	Lazy?
Filter elements	filter(Predicate)	Stream<T>	■
Transform	map(Function)	Stream<R>	■
Flatten	flatMap(Function)	Stream<R>	■
Remove dupes	distinct()	Stream<T>	■
Sort	sorted() / sorted(Comparator)	Stream<T>	■
Debug	peek(Consumer)	Stream<T>	■
Limit size	limit(n)	Stream<T>	■
Skip elements	skip(n)	Stream<T>	■
Collect	collect(Collector)	R	■
Count	count()	long	■
Any match	anyMatch(Predicate)	boolean	■
All match	allMatch(Predicate)	boolean	■
None match	noneMatch(Predicate)	boolean	■
First element	findFirst()	Optional<T>	■
Any element	findAny()	Optional<T>	■
Min/Max	min/max(Comparator)	Optional<T>	■
Fold/Reduce	reduce(identity, op)	T / Optional<T>	■
Side effects	forEach(Consumer)	void	■
To array	toArray()	Object[]	■

Collectors Quick Reference

Collector	Use Case
toList() / toSet()	Basic collection
toMap(k, v)	Build a Map
groupingBy(classifier)	Group → Map<K, List<V>>
groupingBy(c, counting())	Count per group
groupingBy(c, averagingDouble(f))	Average per group
groupingBy(c, summingInt(f))	Sum per group

<code>groupingBy(c, mapping(m, toList()))</code>	Map values per group
<code>partitioningBy(Predicate)</code>	Split into true/false map
<code>joining(sep, prefix, suffix)</code>	Concatenate strings
<code>summarizingInt/Long/Double(f)</code>	Statistics (min/max/avg/sum)
<code>collectingAndThen(c, finisher)</code>	Post-process collected result
<code>counting()</code>	Count elements
<code>teeing(c1, c2, merger)</code>	Combine two collectors (Java 12+)

■ Master these patterns: `groupingBy+counting`, `partitioningBy+mapping`, `flatMap` for nested lists, `reduce` for aggregations, and `Optional` chaining. These cover 90% of all Java Stream interview questions at Indian MNCs.